Programmation Frontend

Lab 3 — useEffect et Cycle de Vie

Chargement automatique, loading et page de détail

ESP/UCAD — 2025–2026

Dr. El Hadji Bassirou TOURÉ
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Informations pratiques

Durée estimée	2h (une séance)
Prérequis	Avoir terminé le Lab 2bis (tri, favoris, immutabilité). Comprendre `useState` et le re-rendu.
Fichiers de départ	Le projet `annuaire-react/` du Lab 2bis
Livrables	Annuaire avec chargement automatique, gestion loading/erreur, et vue de détail
Validation	Les données se chargent au démarrage. Un clic sur une carte affiche le détail.

Objectifs du lab

À l'issue de ce lab, chaque étudiant sera capable de :

1. Expliquer ce qu'est un **effet de bord** (*side effect*) dans React
2. Utiliser `useEffect` pour exécuter du code **au montage** d'un composant
3. Gérer les trois états d'un chargement : **loading**, **erreur** et **données**
4. Utiliser `useEffect` avec une **dépendance** pour réagir à un changement
5. Naviguer entre une vue **liste** et une vue **détail** avec un état

Table des matières

1 Point de départ — Rappel du Lab 2bis **3**
1.1 Le problème . . . 3

2 Partie 1 — Les effets de bord **3**

3 Partie 2 — useEffect : charger au montage **4**
3.1 La syntaxe . . . 4
3.2 Exercice 1 — Supprimer le bouton . . . 4

4 Partie 3 — Gérer le loading et les erreurs **6**
4.1 Le pattern loading / erreur / données . . . 6

4.2 Exercice 2 — Ajouter un état d'erreur 6
4.3 Tester l'erreur 7

5 Partie 4 — Vue de détail avec dépendance 7
5.1 Le principe 7
5.2 Créer `UserDetail.jsx` 8
5.3 Créer `UserDetail.module.css` 9
5.4 Rendre les cartes cliquables 10
5.5 Connecter le tout dans `App.jsx` 11
5.6 Tester 13

6 Bilan — Structure du projet 13

7 Aide-mémoire 14

8 Résumé et conclusion 15

1 Point de départ — Rappel du Lab 2bis

Au Lab 2bis, vous avez compris `useState` en profondeur : le re-rendu, l'immutabilité, la différence entre état et props. L'annuaire a maintenant un tri A-Z/Z-A et un système de favoris. Votre `App.jsx` utilise 5 états :

État	Rôle
`users`	Le tableau des utilisateurs (récupérés par fetch)
`chargement`	Booléen : le fetch est-il en cours ?
`recherche`	Le texte tapé dans la barre de recherche
`triAscendant`	Booléen : tri A→Z ou Z→A ?
`favoris`	Tableau d'identifiants des utilisateurs favoris

Étape 0 — Vérification

Ouvrez le projet `annuaire-react/` dans VS Code. Lancez le serveur :

```
cd annuaire-react
npm run dev
```

Ouvrez `http://localhost:5173`. Cliquez sur « Charger ». Vérifiez que les 10 utilisateurs apparaissent, que le filtre, le tri et les favoris fonctionnent.

1.1 Le problème

Ouvrez la page dans un **nouvel onglet**. Que voyez-vous ? Un écran vide avec un bouton. Il faut cliquer sur « Charger » pour voir les utilisateurs.

Sur n'importe quel site réel, les données sont déjà là quand la page s'affiche. On veut la même chose : les utilisateurs doivent se charger **automatiquement** quand la page s'ouvre.

2 Partie 1 — Les effets de bord

Objectif — 5 minutes

Comprendre pourquoi React a besoin d'un outil séparé pour certaines actions.

Au Lab 2bis, vous avez vu que la fonction d'un composant a un seul rôle : **calculer ce qui doit s'afficher** à partir de l'état et des props. React l'exécute à chaque re-rendu.

Mais parfois, un composant doit aussi faire des choses **en dehors** du rendu :

- Récupérer des données depuis une API (`fetch`)
- Démarrer un timer (`setInterval`)
- Modifier le titre de la page (`document.title`)

Ces actions s'appellent des **effets de bord** (*side effects*). On ne peut pas les mettre directement dans le corps du composant, car elles se déclencheraient à **chaque rendu** (rappelez-vous le `console.log` du Lab 2bis qui s'affichait à chaque clic).

React fournit `useEffect` pour dire : « après le rendu, exécute ceci. »

Analogie : le post-it sur le frigo

Vous vous réveillez le matin (le composant se monte). Vous voyez un post-it sur le frigo : « Acheter du pain ». Vous faites d'abord votre routine (React fait le rendu), puis vous exécutez le post-it (l'effet).

`useEffect` est le post-it de React : « Après le rendu, fais ceci. »

3 Partie 2 — useEffect : charger au montage

Objectif — 20 minutes

Utiliser `useEffect` pour charger les utilisateurs automatiquement et supprimer le bouton.

3.1 La syntaxe

```
import { useState, useEffect } from "react";

useEffect(() => {
  // Code à exécuter après le rendu
  console.log("Le composant a été rendu !");
}, []);
```

`useEffect` prend deux arguments : une **fonction** (le code à exécuter) et un **tableau de dépendances** (quand l'exécuter).

Dépendances	Quand l'effet se déclenche
`[]` (tableau vide)	Une seule fois, au **montage** (quand le composant apparaît)
`[x]`	Au montage + à chaque fois que `x` change
`[x, y]`	Au montage + à chaque fois que `x` ou `y` change
Pas de tableau	À **chaque** rendu (rarement utile)

Pour charger les données au démarrage, on veut que l'effet se déclenche **une seule fois**. On utilise donc `[]`.

3.2 Exercice 1 — Supprimer le bouton

À vous de jouer

Remplacez le bouton « Charger » par un `useEffect` qui charge les données automatiquement.

Quatre changements à faire dans `src/App.jsx` :

1. Ajoutez `useEffect` à l'import : `import { useState, useEffect } from "react";`
2. Déplacez la logique de `chargerUtilisateurs` dans un `useEffect`
3. Initialisez `chargement` à `true` (on charge dès le départ)
4. Supprimez le `<button>` du JSX

Voici le haut du fichier (le reste ne change pas) :

```
import { useState, useEffect } from "react";
import UserCard from "./components/UserCard";
import SearchBar from "./components/SearchBar";
import styles from "./App.module.css";

const App = () => {
  const [users, setUsers] = useState([]);
```

```
const [chargement, setChargement] = useState(true);
const [recherche, setRecherche] = useState("");
const [triAscendant, setTriAscendant] = useState(true);
const [favoris, setFavoris] = useState([]);

useEffect(() => {
  const chargerUtilisateurs = async () => {
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );
      const data = await response.json();
      setUsers(data);
    } catch (error) {
      console.error("Erreur :", error);
    }
    setChargement(false);
  };

  chargerUtilisateurs();
}, []);

// toggleFavori, usersFiltres... inchangés
```

Dans le `return`, supprimez le bloc du bouton « Charger ». Le reste (SearchBar, barre de tri, favoris, cartes) reste identique.

Sauvegardez. Ouvrez la page. Les utilisateurs apparaissent **automatiquement**.

Pourquoi la fonction async est dans le useEffect

`useEffect` ne peut pas être `async` directement. On définit donc une fonction async **à l'intérieur** et on l'appelle immédiatement :

```
useEffect(() => {
  const fetchData = async () => {
    // ... await fetch(...)
  };
  fetchData();
}, []);
```

Avant / après : le chargement

	Lab 2bis	Lab 3
Déclencheur	Bouton « Charger »	Automatique (`useEffect`)
Code	`onClick={fn}`	`useEffect(() => {...}, [])`
Premier affichage	Écran vide	Données visibles

4 Partie 3 — Gérer le loading et les erreurs

Objectif — 15 minutes

Afficher un message de chargement et gérer les erreurs proprement.

4.1 Le pattern loading / erreur / données

Quand on récupère des données depuis une API, trois situations sont possibles :

Situation	Ce qui se passe	Ce qu'on affiche
Loading	La requête est en cours	« Chargement en cours... »
Erreur	La requête a échoué	« Impossible de charger »
Données	La requête a réussi	La liste des utilisateurs

Ce pattern est **universel** : vous le retrouverez dans chaque application qui consomme une API.

4.2 Exercice 2 — Ajouter un état d'erreur

À vous de jouer

Ajoutez un état `erreur` et affichez un message conditionnel selon la situation.

Ajoutez l'état et modifiez le `useEffect` :

```
const [erreur, setErreur] = useState(null);

useEffect(() => {
  const chargerUtilisateurs = async () => {
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );
      if (!response.ok) {
        throw new Error("Erreur serveur");
      }
      const data = await response.json();
      setUsers(data);
    } catch (error) {
      setErreur(error.message);
    }
    setChargement(false);
  };

  chargerUtilisateurs();
}, []);
```

La ligne `if (!response.ok)` vérifie le code HTTP. Si le serveur répond avec une erreur (404, 500...), on lance une exception.

Ajoutez deux *early returns* **avant** le `return` principal :

```
if (chargement) {
  return (
    <div className={styles.container}>
```

```
      <p>Chargement en cours...</p>
    </div>
  );
}

if (erreur) {
  return (
    <div className={styles.container}>
      <p className={styles.erreur}>
        Erreur : {erreur}
      </p>
    </div>
  );
}

// return principal (la liste) inchangé
```

Ajoutez la classe dans `src/App.module.css` :

```
.erreur {
  color: #c0392b;
  font-weight: bold;
  padding: 1rem;
  background: #fdecea;
  border-radius: 6px;
}
```

Le pattern early return

1. **Si loading** → afficher « Chargement... » et `return`.
2. **Si erreur** → afficher le message d'erreur et `return`.
3. **Sinon** → afficher les données.

On traite d'abord les cas particuliers, puis le cas général.

4.3 Tester l'erreur

Pour vérifier, changez temporairement l'URL en `/FAUX`. Le message d'erreur rouge apparaît. Remettez `/users` après le test.

5 Partie 4 — Vue de détail avec dépendance

Objectif — 20 minutes

Cliquer sur une carte pour afficher les informations complètes d'un utilisateur.

5.1 Le principe

On veut deux vues : la **liste** (ce qu'on a) et le **détail** (informations complètes). Un état `selectedUserId` contrôle la vue active : `null` = liste, un identifiant = détail.

Pourquoi pas un vrai routeur ?

Le routing (changement d'URL, bouton retour du navigateur) est un sujet à part entière. On l'apprendra au **Lab 4**. Aujourd'hui, on se concentre sur **une seule chose** : `useEffect`. La navigation par état est une solution temporaire.

5.2 Créer `UserDetail.jsx`

Créez `src/components/UserDetail.jsx` :

```
import { useState, useEffect } from "react";
import styles from "./UserDetail.module.css";

const UserDetail = ({ userId, onRetour }) => {
  const [user, setUser] = useState(null);
  const [chargement, setChargement] = useState(true);
  const [erreur, setErreur] = useState(null);

  useEffect(() => {
    const chargerDetail = async () => {
      setChargement(true);
      setErreur(null);
      try {
        const response = await fetch(
          `https://jsonplaceholder.typicode.com/users/${userId}`
        );
        if (!response.ok) {
          throw new Error("Utilisateur introuvable");
        }
        const data = await response.json();
        setUser(data);
      } catch (error) {
        setErreur(error.message);
      }
      setChargement(false);
    };

    chargerDetail();
  }, [userId]);

  if (chargement) return <p>Chargement du profil...</p>;

  if (erreur) {
    return (
      <div>
        <p className={styles.erreur}>
          Erreur : {erreur}
        </p>
        <button className={styles.retour}
                onClick={onRetour}>
          Retour à la liste
        </button>
      </div>
    );
  }

  return (
```

```
    <div className={styles.detail}>
      <button className={styles.retour}
              onClick={onRetour}>
        Retour à la liste
      </button>
      <h2 className={styles.nom}>{user.name}</h2>
      <table className={styles.tableau}>
        <tbody>
          <tr>
            <td className={styles.label}>Email</td>
            <td>{user.email}</td>
          </tr>
          <tr>
            <td className={styles.label}>Téléphone</td>
            <td>{user.phone}</td>
          </tr>
          <tr>
            <td className={styles.label}>Site web</td>
            <td>{user.website}</td>
          </tr>
          <tr>
            <td className={styles.label}>Entreprise</td>
            <td>{user.company.name}</td>
          </tr>
          <tr>
            <td className={styles.label}>Ville</td>
            <td>{user.address.city}</td>
          </tr>
        </tbody>
      </table>
    </div>
  );
};

export default UserDetail;
```

Deux choses à observer :

1. Le `useEffect` utilise `[userId]` comme dépendance. Si `userId` change, l'effet se re-déclenche et charge les nouvelles données.
2. Le composant applique le même pattern loading/erreur/données que `App`.

5.3 Créer `UserDetail.module.css`

Créez `src/components/UserDetail.module.css` :

```
.detail { padding: 1rem 0; }

.retour {
  padding: 0.4rem 1rem;
  font-size: 0.9rem;
  cursor: pointer;
  border: 1px solid #ccc;
  border-radius: 6px;
  background: #f9f9f9;
  margin-bottom: 1rem;
}
.retour:hover { background: #eee; }
```

```
.nom {
  margin: 0.5rem 0 1rem 0;
  color: #333;
}

.tableau { width: 100%; border-collapse: collapse; }
.tableau td {
  padding: 0.5rem 0.75rem;
  border-bottom: 1px solid #eee;
}
.label {
  font-weight: bold;
  color: #555;
  width: 120px;
}
.erreur { color: #c0392b; font-weight: bold; }
```

5.4 Rendre les cartes cliquables

Modifiez `src/components/UserCard.jsx` pour ajouter une prop `onClick` :

```
import styles from "./UserCard.module.css";

const UserCard = ({
  user, estFavori, onToggleFavori, onClick
}) => {
  return (
    <div className={styles.card}
         onClick={() => onClick(user.id)}>
      <div className={styles.header}>
        <h3 className={styles.nom}>{user.name}</h3>
        <button className={styles.etoile}
                onClick={(e) => {
                  e.stopPropagation();
                  onToggleFavori(user.id);
                }}>
          {estFavori ? "\u2605" : "\u2606"}
        </button>
      </div>
      <p className={styles.info}>
        Email : {user.email}
      </p>
      <p className={styles.info}>
        Entreprise : {user.company.name}
      </p>
    </div>
  );
};

export default UserCard;
```

`e.stopPropagation()` sur le bouton étoile

Sans `e.stopPropagation()`, cliquer sur l'étoile déclencherait **aussi** le `onClick` de la carte (l'étoile est à l'intérieur). `stopPropagation` empêche l'événement de remonter. Le clic sur

l'étoile reste un toggle favori, le clic sur le reste de la carte ouvre le détail.

Ajoutez `cursor: pointer;` dans `.card` de `UserCard.module.css` si ce n'est pas déjà fait.

5.5 Connecter le tout dans `App.jsx`

Ajoutez l'état `selectedUserId`, l'import de `UserDetail`, et un early return pour la vue détail. Voici le fichier complet :

```
import { useState, useEffect } from "react";
import UserCard from "./components/UserCard";
import UserDetail from "./components/UserDetail";
import SearchBar from "./components/SearchBar";
import styles from "./App.module.css";

const App = () => {
  const [users, setUsers] = useState([]);
  const [chargement, setChargement] = useState(true);
  const [erreur, setErreur] = useState(null);
  const [recherche, setRecherche] = useState("");
  const [triAscendant, setTriAscendant] = useState(true);
  const [favoris, setFavoris] = useState([]);
  const [selectedUserId, setSelectedUserId] =
    useState(null);

  useEffect(() => {
    const chargerUtilisateurs = async () => {
      try {
        const response = await fetch(
          "https://jsonplaceholder.typicode.com/users"
        );
        if (!response.ok) {
          throw new Error("Erreur serveur");
        }
        const data = await response.json();
        setUsers(data);
      } catch (error) {
        setErreur(error.message);
      }
      setChargement(false);
    };

    chargerUtilisateurs();
  }, []);

  const toggleFavori = (userId) => {
    if (favoris.includes(userId)) {
      setFavoris(favoris.filter(id => id !== userId));
    } else {
      setFavoris([...favoris, userId]);
    }
  };

  if (chargement) {
    return (
      <div className={styles.container}>
        <p>Chargement en cours...</p>
```

```
      </div>
    );
  }

  if (erreur) {
    return (
      <div className={styles.container}>
        <p className={styles.erreur}>
          Erreur : {erreur}
        </p>
      </div>
    );
  }

  if (selectedUserId) {
    return (
      <div className={styles.container}>
        <h1 className={styles.titre}>
          Annuaire des utilisateurs
        </h1>
        <UserDetail
          userId={selectedUserId}
          onRetour={() => setSelectedUserId(null)}
        />
      </div>
    );
  }

  const usersFiltres = users
    .filter(user =>
      user.name.toLowerCase().includes(
        recherche.toLowerCase()
      )
    )
    .sort((a, b) => {
      if (triAscendant) {
        return a.name.localeCompare(b.name);
      }
      return b.name.localeCompare(a.name);
    });

  return (
    <div className={styles.container}>
      <h1 className={styles.titre}>
        Annuaire des utilisateurs
      </h1>

      <SearchBar
        recherche={recherche}
        onRecherche={setRecherche}
      />

      <div className={styles.barre}>
        <p className={styles.compteur}>
          {usersFiltres.length} utilisateur(s)
        </p>
        <button className={styles.boutonTri}
                onClick={() =>
```

```
                  setTriAscendant(!triAscendant)
                }>
          Tri : {triAscendant ? "A → Z" : "Z → A"}
        </button>
      </div>

      {favoris.length > 0 && (
        <p className={styles.favoris}>
          {favoris.length} favori(s)
        </p>
      )}

      {usersFiltres.map(user => (
        <UserCard
          key={user.id}
          user={user}
          estFavori={favoris.includes(user.id)}
          onToggleFavori={toggleFavori}
          onClick={setSelectedUserId}
        />
      ))}
    </div>
  );
};

export default App;
```

5.6 Tester

Sauvegardez tous les fichiers. Vérifiez :

1. Les utilisateurs se chargent **automatiquement**.
2. Cliquez sur une carte → le détail s'affiche.
3. Cliquez sur « Retour à la liste » → la liste réapparaît.
4. Les étoiles de favoris fonctionnent toujours (l'étoile ne déclenche pas la vue détail).
5. Le filtre et le tri fonctionnent toujours.

6 Bilan — Structure du projet

```
annuaire-react/
  src/
    App.jsx
    App.module.css
    components/
      UserCard.jsx
      UserCard.module.css
      UserDetail.jsx        <-- nouveau
      UserDetail.module.css  <-- nouveau
      SearchBar.jsx
      SearchBar.module.css
    main.jsx
  index.html
  package.json
```

Deux useEffect — deux rôles

Composant	Dépendances	Rôle
`App`	`[]`	Charger la liste une seule fois au démarrage
`UserDetail`	`[userId]`	Charger le détail quand l'id change

Avant / après : Lab 2bis vs Lab 3

	Lab 2bis	Lab 3
Chargement	Bouton « Charger »	Automatique (`useEffect`)
Loading	Texte dans le bouton	Écran dédié (early return)
Erreurs	`console.error`	Message visible à l'écran
Détail	Aucun	Vue complète au clic
Navigation	Aucune	Liste ↔ Détail par état

7 Aide-mémoire

Concept	Syntaxe / Exemple
Importer useEffect	`import { useState, useEffect } from "react";`
Au montage seulement	`useEffect(() => { ... }, []);`
Quand `x` change	`useEffect(() => { ... }, [x]);`
À chaque rendu	`useEffect(() => { ... });` (rare)
Async dans useEffect	Définir une fonction async à l'intérieur, puis l'appeler
Vérifier la réponse	`if (!response.ok) throw new Error(...);`
Early return (loading)	`if (chargement) return <p>...</p>;`
Early return (erreur)	`if (erreur) return <p>...</p>;`
Navigation par état	`const [id, setId] = useState(null);`
Vue conditionnelle	`if (id) return <Detail />;`
Bloquer la propagation	`e.stopPropagation()`

8 Résumé et conclusion

Dans ce lab, vous avez :

1. Compris les **effets de bord** : des actions en dehors du rendu (fetch, timers).
2. Supprimé le bouton et chargé les données **automatiquement** avec `useEffect(…, [])`.
3. Mis en place le pattern **loading / erreur / données** avec des *early returns*.
4. Créé un composant `UserDetail` avec `useEffect` et une **dépendance** `[userId]`.
5. Navigué entre liste et détail grâce à un état `selectedUserId`.

Étape	Statut	Ce que vous savez faire
DOM et événements	✓	Sélecteurs, événements, modification du DOM
Fonctions JS	✓	Déclaration, expression, fléchée
Asynchrone	✓	`fetch`, `async/await`
React — les bases	✓	Composant, JSX, Vite, `.map()`
Composants et props	✓	Découpage, props, CSS Modules
État React	✓	`useState`, re-rendu, immutabilité
useEffect	✓	Montage, dépendances, loading/erreur
Formulaires et routing	→ Lab 4	Inputs contrôlés, React Router

Et ensuite ?

L'annuaire charge automatiquement, gère les erreurs et permet de voir le détail d'un utilisateur. Mais la navigation est gérée par un état : l'URL ne change pas et le bouton retour du navigateur ne fonctionne pas.
Dans le **Lab 4**, vous installerez **React Router** pour créer une vraie application multi-pages. Chaque vue aura sa propre URL (`/users`, `/users/3`), et vous ajouterez un **formulaire contrôlé** pour saisir de nouveaux utilisateurs.

Exercice bonus — Pour aller plus loin

Bonus : un hook personnalisé useFetch

Extrayez la logique fetch/loading/erreur dans un hook réutilisable. Créez `src/hooks/useFetch.js` :

```
import { useState, useEffect } from "react";

const useFetch = (url) => {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      setLoading(true);
      setError(null);
      try {
        const response = await fetch(url);
        if (!response.ok) throw new Error("Erreur");
        const result = await response.json();
        setData(result);
      } catch (err) {
```

```
        setError(err.message);
      }
      setLoading(false);
    };

    fetchData();
  }, [url]);

  return { data, loading, error };
};

export default useFetch;
```

Utilisation dans `UserDetail.jsx` :

```
import useFetch from "../hooks/useFetch";

const UserDetail = ({ userId, onRetour }) => {
  const { data: user, loading, error } = useFetch(
    `https://jsonplaceholder.typicode.com/users/${userId}`
  );

  if (loading) return <p>Chargement...</p>;
  if (error) return <p>Erreur : {error}</p>;

  return (
    <div>
      <h2>{user.name}</h2>
      {/* ... le reste du JSX */}
    </div>
  );
};
```

Le hook encapsule le pattern loading/erreur/données. Il est réutilisable dans n'importe quel composant qui charge des données.

Lab 3 — useEffect et Cycle de Vie — Terminé ✓

Vous savez charger des données automatiquement,
gérer les états de chargement et afficher une vue de détail.

Prochaine étape : React Router et formulaires contrôlés.

useEffect = le post-it de React.